

Лабораторная работа №3

Построение сцены дополненной реальности в реальном времени с использованием данных с игрового сервера

Введение

Современные системы дополненной реальности (Augmented Reality, AR) предполагают наличие специализированного устройства отображения, обеспечивающего совмещение синтетической графики с изображением реального мира. В предыдущих лабораторных работах задача оценки положения камеры решалась с помощью методов компьютерного зрения: через калибровку камеры (лабораторная работа №1) и оценку позы по искусственным маркерам (лабораторная работа №2). В обоих случаях вычисление пространственных преобразований выполнялось на стороне разработчика, а вывод формировался на двумерном экране.

В настоящей работе акцент смещается с реализации низкоуровневого CV-стека на использование готовой платформы дополненной реальности на базе шлема Meta Quest 3. Шлем берёт на себя задачи трекинга, реконструкции пространства и отрисовки стереоскопического изображения, освобождая разработчика от необходимости вручную решать задачи SLAM, оценки позы и калибровки. Внимание разработчика смещается на уровень выше: на формирование сцены, привязку виртуальных объектов к реальному пространству и интеграцию с внешними источниками данных.

В качестве источника данных в данной работе используется сервер игры Minecraft. Выбор обусловлен тем, что мир Minecraft имеет регулярную воксельную структуру, естественным образом отображаемую в виде набора кубических примитивов; при этом данные о состоянии мира доступны через серверный API, что позволяет построить полноценный клиент-серверный конвейер передачи данных в реальном времени.

Цель работы — разработать систему дополненной реальности, в которой клиент-приложение для Meta Quest 3 в реальном времени получает данные о выбранной области мира Minecraft с серверного плагина по протоколу WebSocket и отображает эти данные в виде воксельной 3D-сцены, привязанной к поверхности стола в режиме passthrough-AR.

Содержательно работа охватывает три предметных слоя:

- аппаратно-программная платформа Meta Quest 3 и её SDK;

- серверная сторона — плагин для Minecraft-сервера, экспортирующий состояние мира;
- сетевой и рендеринг-стек — передача данных по WebSocket и их воксельная визуализация.

После успешного выполнения работы формируется представление о принципах функционирования современных AR-устройств класса Mixed Reality, об устройстве их SDK, о том, как привязывать виртуальное содержимое к реальной геометрии помещения, а также о практической интеграции AR-клиента с произвольным сетевым источником данных.

Архитектура шлема Meta Quest 3

Meta Quest 3 относится к классу автономных (standalone) шлемов смешанной реальности. В отличие от устройств, требующих подключения к ПК, шлем содержит все вычислительные и сенсорные подсистемы внутри корпуса и представляет собой самодостаточный компьютер на базе ОС Android (Horizon OS, ранее Quest OS), исполняющий приложения непосредственно на устройстве.

Вычислительная платформа

В основе устройства лежит мобильная система-на-кристалле **Qualcomm Snapdragon XR2 Gen 2**, специализированный SoC для XR-устройств, включающий:

- многоядерный CPU на архитектуре ARMv9;
- GPU Adreno с поддержкой Vulkan и аппаратного фовеированного рендеринга;
- DSP, выделенные для обработки сенсорных данных;
- аппаратные блоки кодирования/декодирования видео для passthrough.

Объём оперативной памяти составляет 8 ГБ, встроенной — 128 или 512 ГБ. Эти параметры существенно ограничивают размеры рабочей сцены, что особенно важно при воксельном рендеринге: количество одновременно отрисовываемых блоков прямо влияет на потребление памяти и пропускную способность шины.

Подсистема отображения

Шлем оснащён двумя дисплеями типа LCD с рапсак-оптикой. Разрешение каждого дисплея составляет 2064×2208 пикселей на глаз, частота обновления — 90 Гц (с поддержкой 120 Гц в экспериментальных режимах). Использование рапсак-оптики

позволяет уменьшить толщину передней части шлема и снизить вес по сравнению с френелевыми линзами Quest 2.

Поле зрения (FOV) составляет приблизительно 110° по горизонтали и 96° по вертикали, межзрачковое расстояние (IPD) регулируется в диапазоне от 53 до 75 мм, что обеспечивает корректное стереоскопическое восприятие для большинства пользователей.

Сенсорная подсистема

Для пространственной ориентации и обеспечения passthrough-режима шлем оснащён следующим набором сенсоров:

- четыре монохромные камеры по углам корпуса — используются для inside-out трекинга и распознавания рук;
- две цветные RGB-камеры высокого разрешения, обеспечивающие цветной видеопроход (passthrough);
- датчик глубины (depth sensor) на основе структурированного света, используемый для построения карты окружения;
- инерциальный измерительный модуль (IMU): акселерометр, гироскоп, магнитометр.

Принципиальным отличием Quest 3 от предыдущих моделей является наличие выделенного датчика глубины, что позволяет в реальном времени получать карту дистанций до объектов сцены и использовать её для маскирования и взаимодействия виртуальных объектов с физической средой (depth occlusion).

Принципы работы Meta Quest 3

Корректное совмещение виртуального содержимого с реальным миром требует, чтобы шлем непрерывно знал собственное положение и ориентацию в пространстве, а также имел представление о геометрии окружения. Эти задачи решаются на уровне системного ПО Quest и предоставляются приложению уже в виде готовых поз и моделей сцены.

Inside-out трекинг

Quest 3 использует так называемый **inside-out tracking** — определение позы шлема относительно внешнего мира исключительно за счёт сенсоров, расположенных на самом шлеме, без внешних базовых станций.

Алгоритм трекинга относится к классу **Visual-Inertial SLAM** (Simultaneous Localization and Mapping) и объединяет:

- визуальные особенности (feature points), извлекаемые из изображений камер;
- инерциальные данные с IMU, обеспечивающие высокую частоту обновления и устойчивость к быстрым движениям;
- построение разреженной карты особых точек окружения и одновременное оценивание положения шлема в этой карте.

Результатом является поза шлема в системе координат отслеживаемого пространства (tracking space), обновляемая с частотой кадра и доступная приложению как 6 степеней свободы (6DoF). Аналогичный механизм применяется и для контроллеров Touch Plus, поза которых вычисляется по их видимости в камерах шлема и данным их собственных IMU.

Системы координат

В Meta XR SDK выделяются несколько систем координат, которые необходимо различать при размещении виртуальных объектов:

- **Tracking space** — глобальная система координат сессии, в которой ведётся трекинг шлема. Не привязана к содержимому сцены Unity напрямую.
- **Local floor (Floor-Level)** — система с началом в точке проекции шлема на пол в момент калибровки и осью Y вверх. Используется по умолчанию для приложений со “стоящим” пользователем.
- **Stage** — система, привязанная к зоне Guardian/Boundary, заданной пользователем.
- **Local pose шлема** — собственная локальная система камеры, в которой ведётся отрисовка стереопары.

Соглашение об осях соответствует Unity: ось X направлена вправо, ось Y — вверх, ось Z — вперёд. Размеры заданы в метрах, что особенно важно при сопоставлении масштаба виртуальной сцены с физическим миром.

Цветной passthrough

Под passthrough понимается отображение видео с внешних камер шлема на экранах перед глазами пользователя, что создаёт эффект “прозрачности” шлема. Quest 3, в отличие от Quest 2, поддерживает полноцветный passthrough благодаря выделенным RGB-камерам высокого разрешения.

С точки зрения приложения passthrough активируется через механизм **compositor layers**: системный композитор шлема накладывает видеоизображение в качестве фонового слоя, поверх которого отрисовывается виртуальная сцена. Приложение управляет лишь тем, какие области экрана останутся прозрачными для passthrough.

Поскольку RGB-камеры пространственно смещены относительно глаз пользователя, изображение проходит репроекцию с учётом текущей карты глубины окружения. Эта операция полностью выполняется системным ПО шлема и для приложения остаётся прозрачной.

Реконструкция сцены и пространственные якоря

Для размещения виртуальных объектов на конкретной поверхности (например, на столе) недостаточно знать позу шлема — необходимо знать геометрию помещения. Эта задача решается двумя механизмами:

- **Scene Model** — статическая модель помещения, формируемая пользователем в процессе *Room Setup*. В неё входят размеченные плоскости (стены, пол, потолок), мебель (столы, диваны), а также точечные объекты. Модель хранится на уровне ОС шлема и доступна всем приложениям.
- **Spatial anchors** — “якоря”, привязанные к конкретным точкам реального пространства. Позиция якоря корректируется системой при дрейфе SLAM и сохраняется между сессиями.

В рамках настоящей работы стол используется в качестве плоскости-носителя для отображения чанков мира Minecraft. Это означает, что приложение должно либо извлечь плоскость стола из Scene Model, либо позволить пользователю задать якорь вручную в начале сессии.

Meta XR SDK: организация и ключевые компоненты

Разработка приложений для Meta Quest 3 ведётся преимущественно в среде Unity с использованием набора пакетов **Meta XR SDK**. SDK представляет собой расширение стандартного OpenXR-стека Unity, добавляющее специфичные для платформы Quest возможности (passthrough, scene understanding, платформенные сервисы, голосовой ввод и т. д.).

Scene Understanding и привязка к плоскости стола

Реалистичное размещение виртуальных объектов в физическом мире требует знания о геометрии окружения. Для этого Meta XR SDK предоставляет два уровня абстракции: уровень Scene Model и уровень spatial anchors.

Scene Model

Scene Model — это размеченная пользователем модель помещения, формируемая в системной утилите Room Setup шлема. Пользователь указывает положение пола,

потолка, стен, а также добавляет описания крупных объектов мебели. Каждому такому объекту назначается семантическая метка (FLOOR, CEILING, WALL_FACE, TABLE, COUCH, DESK и т. д.) и геометрическое описание в виде ограниченной плоскости или ограниченного объёма (*volume*).

С точки зрения приложения Scene Model доступна как набор *anchor*-объектов, каждый из которых имеет:

- уникальный идентификатор;
- позу в *tracking space*;
- набор семантических меток;
- геометрический компонент (2D-полигон или 3D-объём).

Запрос модели и поиск элементов с нужной семантикой в MRUK выполняется через высокоуровневый API: запрашиваются все объекты, имеющие метку, например, TABLE, и из них выбирается ближайший к пользователю либо тот, на который пользователь смотрит.

Spatial anchors

Spatial anchor — это точка в пространстве, поза которой непрерывно корректируется системой по мере уточнения карты окружения. В отличие от обычной позы в *TrackingSpace*, поза якоря устойчива к дрейфу SLAM и может сохраняться между сессиями (*persistent anchors*).

Ключевое свойство якорей: виртуальный объект, спаренный с якорем, остаётся “приклеенным” к конкретной точке физического пространства независимо от того, как пользователь перемещался между сессиями.

В рамках задачи о размещении карты Minecraft на столе якорь выполняет следующую функцию: можно либо использовать готовую плоскость TABLE из Scene Model, либо позволить пользователю в начале сессии указать центр будущей карты (например, наведя контроллер на стол и нажав кнопку), создав в этом месте новый якорь.

Привязка содержимого

После того как якорь стола получен, его поза $T_{\text{уд}}$ задаёт локальную систему координат сцены Minecraft в физическом пространстве. Все воксельные блоки, рендеримые приложением, размещаются как дочерние объекты якоря, и их позиции выражаются в **локальной системе якоря**, а не в мировой системе Unity.

При получении нового кадра данных от сервера приложению достаточно обновить геометрию дочерних объектов; глобальная привязка к столу сохраняется автоматически за счёт спарентивания с якорем. Этот приём позволяет полностью отделить логику работы со Scene Model от логики работы с сетевыми данными.

Серверная часть: плагин Minecraft-сервера

Для того чтобы AR-клиент мог получать данные о состоянии мира Minecraft, на стороне сервера необходим компонент, который:

- имеет доступ к внутреннему состоянию мира (блоки, существа);
- способен сериализовать выбранную область в компактное представление;
- принимает входящие сетевые подключения от внешних клиентов;
- передаёт данные с приемлемой задержкой и поддерживает обновления при изменении мира.

Платформа плагина

В качестве серверной платформы используется одна из распространённых реализаций Minecraft-сервера, поддерживающих систему плагинов и предоставляющих программный доступ к состоянию мира. Конкретный выбор платформы (Spigot, Paper или иной совместимый форк) и способ организации плагина определяются самостоятельно.

Доступ к данным мира

Мир Minecraft имеет регулярную структуру и логически разбит на чанки — вертикальные колонны блоков фиксированного горизонтального размера. Плагину доступно содержимое чанков (типы блоков, их положение) и информация о существах, находящихся в выбранной области: их тип, координаты, ориентация и состояние. Способ извлечения этих данных определяется выбранным API сервера; при работе с большими областями необходимо учитывать, что обращение к состоянию мира должно выполняться так, чтобы не блокировать основной игровой поток.

Стратегия экспорта данных

С точки зрения протокола разумно различать два типа сообщений:

- **Snapshot** — полный снимок выбранной области, отправляемый при первом подключении клиента, а также при существенных изменениях (например, при смене области интереса);
- **Diff** — инкрементальное обновление, содержащее только изменившиеся блоки и обновлённые позиции существ.

Источниками изменений на стороне плагина выступают игровые события, связанные с установкой и разрушением блоков, а также с перемещением существ. Изменения целесообразно агрегировать за короткий интервал времени и рассылать накопленный diff всем подключённым клиентам. Конкретный формат сообщений и стратегия агрегирования определяются самостоятельно.

WebSocket как транспорт

Передача данных между плагином и AR-клиентом требует протокола, удовлетворяющего нескольким требованиям:

- низкая задержка — обновления должны достигать клиента в темпе, сопоставимом с темпом изменений мира (не хуже нескольких десятков миллисекунд);
- полная двусторонность — сервер должен иметь возможность инициативно отправлять обновления клиенту, не дожидаясь запроса;
- длительное соединение — установка соединения происходит однократно, после чего обмен идёт без накладных расходов на повторные рукопожатия;
- совместимость с типовыми сетевыми условиями — работа поверх стандартных портов, прохождение прокси и NAT.

Этим требованиям удовлетворяет протокол **WebSocket** (RFC 6455). По сути WebSocket — это надстройка над TCP, обеспечивающая семантику обмена дискретными сообщениями (фреймами) поверх постоянного соединения.

Сообщения WebSocket передаются в виде дискретных фреймов и могут быть как текстовыми, так и бинарными; выбор формата сериализации, схемы сообщений и стратегии разделения данных на потоки определяется самостоятельно исходя из поставленных требований.

Конкретный выбор библиотек на стороне сервера и клиента, а также организация асинхронного приёма сообщений и их передачи в основной поток Unity также остаются на усмотрение разработчика.

Воксельный рендеринг и отсечение невидимых граней

Полученная от сервера область мира представляет собой трёхмерный массив идентификаторов блоков размера $16 \times 384 \times 16$ на чанк (для нескольких чанков — кратно). Прямое отображение каждого непустого блока как самостоятельного объекта быстро приводит к недопустимому числу полигонов и draw call'ов.

Наивный рендеринг кубов

В простейшем (наивном) подходе каждый непустой блок отображается как куб с шестью гранями. Для одного чанка $16 \times 16 \times 384 = 98304$ блоков; даже при 50% заполненности это около 50 тыс. кубов на чанк, или порядка 600 тыс. кубов для области 4×4 чанка — что заведомо непрактично без оптимизаций.

Отсечение внутренних граней

Самая простая, но при этом достаточно эффективная оптимизация — **рендеринг только внешне видимых граней**. Грань блока в направлении $\vec{n}$ отрисовывается тогда и только тогда, когда соседний блок в этом направлении **отсутствует** (является воздухом или находится за пределами области).

Формально: для каждого блока B с координатами (x, y, z) и для каждого из шести направлений $\vec{n}_i \in \{\pm\vec{e}_x, \pm\vec{e}_y, \pm\vec{e}_z\}$ грань $B \cdot \vec{n}_i$ добавляется в итоговый меш, если выполнено условие:

$$\text{block}(x + n_{i,x}, y + n_{i,y}, z + n_{i,z}) = \text{air}. \quad (1)$$

Геометрический смысл прост: грани между двумя плотными блоками не видны ни с одного ракурса и поэтому не подлежат отрисовке.

Эффект оптимизации

Для типичного ландшафта Minecraft с большими массивами однородной породы под землёй доля внешне видимых граней составляет, как правило, единицы процентов от общего числа граней всех блоков. Это означает, что отсечение внутренних граней снижает число полигонов на один–два порядка и обычно достаточно для устойчивого рендеринга нескольких чанков на Quest 3 в наивной реализации без перехода к более сложным методам типа greedy meshing.

Построение меша в Unity

В Unity рендеринг такой геометрии реализуется через прямое формирование объекта **Mesh**: собираются массивы вершин (**vertices**), нормалей (**normals**), UV-координат (**uv**) и индексов треугольников (**triangles**), после чего они присваиваются компоненту **MeshFilter**. Желательно объединять блоки одного типа в один меш, чтобы минимизировать число draw call'ов; разделение по типам блоков обычно соответствует разделению по материалам/атласам текстур.

При получении diff-обновления от сервера полное пересчитывание меша всей области избыточно. Разумной стратегией является разбиение области на под-блоки (например, $16 \times 16 \times 16$, как чанк-секция в самом Minecraft) и пересчёт меша только тех под-блоков, в которых произошли изменения.

Освещение и материалы в AR

Поскольку сцена рендерится в режиме passthrough, освещение виртуальных объектов должно выглядеть согласованно с реальным освещением комнаты. Полноценная реконструкция освещения окружения в рамках настоящей работы не требуется; достаточно использовать неярное равномерное освещение с лёгкой направленной составляющей, чтобы воксельные грани оставались различимыми, но не выглядели чрезмерно “пластиковыми”.

Для текстурирования удобно использовать единый атлас текстур блоков; UV-координаты вершин при построении меша вычисляются из идентификатора блока и направления грани.

Существа Minecraft в AR-сцене

Помимо статических блоков, в области интереса могут находиться **существа** (*entities*): мобы, животные, игроки. Их представление в AR-клиенте отличается от представления блоков: существа — это подвижные объекты переменной геометрии, чьи позиции обновляются значительно чаще, чем содержимое блоков.

Передача данных о существах

Для каждого существа в передаваемом сообщении достаточно следующего набора полей:

- уникальный идентификатор;
- тип (например, ZOMBIE, COW, VILLAGER);
- позиция в координатах мира Minecraft;
- поворот тела (yaw) и головы;
- состояние, влияющее на отображение (опционально — стоит/движется/мёртв).

Обновления позиций существ удобно отправлять отдельным потоком сообщений, чем обновления блоков, поскольку их частота принципиально выше: позиции мобов меняются каждые несколько игровых тиков, тогда как блоки изменяются эпизодически.

Отображение в Unity

В рамках настоящей работы существа отображаются **без анимации скелетной деформации**. Каждому типу существа соответствует один статический 3D-меш (либо набор статических мешей для разных состояний). При получении сообщения клиент:

1. проверяет, существует ли уже игровой объект с таким идентификатором;
2. если нет — инстанцирует префаб соответствующего типа как дочерний объект якоря стола;
3. обновляет позицию и поворот объекта согласно полученным данным;
4. существ, отсутствующих в последнем снимке более длительное время, удаляет.

Согласование систем координат

Координаты существ, как и координаты блоков, приходят в системе мира Minecraft. Перевод в локальную систему якоря стола требует двух преобразований:

- смещение начала координат в выбранную опорную точку области (например, центр первого чанка);
- масштабирование с коэффициентом s , переводящим один блок Minecraft (1 м в игровом мире) в физический размер блока на столе (например, 2–3 см).

Коэффициент масштабирования s должен быть единым для всех видов содержимого — блоков и существ — иначе нарушится пространственная согласованность сцены.

Задание

Необходимо разработать систему дополненной реальности, состоящую из двух взаимодействующих компонентов:

- **серверного плагина** для Minecraft-сервера, экспортирующего данные о выбранной области мира;
- **клиентского приложения** для шлема Meta Quest 3, подключающегося к серверу и отображающего полученные данные в режиме passthrough-AR.

Система должна обеспечивать следующее:

- плагин предоставляет доступ к области размером в несколько чанков и передаёт состояние блоков и существ в этой области;
- клиент устанавливает соединение с плагином и получает первоначальный снимок области;
- виртуальная сцена привязывается к плоскости стола в аудитории;
- масштаб сцены подобран так, чтобы область удобно располагалась на столе или может быть подобран пользователем динамически;

- существа отображаются статическими мешами без анимации скелетной деформации;
- клиент применяет инкрементальные обновления данных по мере их поступления;
- производительность системы должна быть достаточной, чтобы не вызывать болезненных ощущений у пользователей во время использования.

Архитектура решения, формат сетевого протокола и способы реализации отдельных подсистем определяются самостоятельно. При сдаче работы необходимо обосновать ключевые принятые решения.

Контрольные вопросы

1. Почему для определения позы шлема недостаточно одних только камер или одного только IMU, и что даёт их объединение в Visual-Inertial SLAM?
2. Что такое дрейф в SLAM, откуда он возникает и как механизм spatial anchor его компенсирует?
3. Зачем в системе одновременно существуют несколько систем координат (tracking space, floor-level, локальная система якоря) и какую задачу решает каждая из них?
4. Зачем в AR-приложении нужен явный этап «выбора плоскости стола» (через Scene Model или ручную постановку якоря) до начала загрузки и отрисовки контента, а не сразу после старта приложения?
5. Почему между приходом данных из сети и появлением соответствующих игровых объектов в сцене должна быть промежуточная очередь/буфер, а не прямое создание объектов в коллбэке WebSocket?
6. Что в архитектуре приложения должно произойти при потере и повторном установлении WebSocket-соединения, и почему нельзя просто продолжить применять входящие diff-апдейты как ни в чём не бывало?
7. Почему для задачи «клиент непрерывно получает обновления состояния мира» WebSocket подходит лучше, чем периодический HTTP-поллинг или long polling, даже если последние тоже технически решают задачу?
8. В чём смысл архитектурного разделения сообщений на snapshot и diff, и какую проблему решает наличие именно двух типов, а не только diff'ов с самого начала сессии?
9. Почему сервер должен агрегировать изменения мира за короткий интервал (десятки миллисекунд) и слать их пачкой, а не отправлять по одному сообщению на каждое изменение блока?

10. Какие гарантии порядка и доставки даёт WebSocket «из коробки», и каких гарантий он не даёт, что важно учесть при проектировании протокола diff-обновлений?
11. Почему стоит выбирать между бинарным и текстовым (JSON) форматом сообщений на разных стадиях разработки, и какие соображения определяют выбор в финальной версии?
12. Почему построение игровых объектов нельзя выполнять в потоке, в котором ведётся приём WebSocket-сообщений?
13. Какие преобразования координат необходимо выполнить при размещении блока с координатами Minecraft (x, y, z) на физическом столе?

Данный список вопросов является базовым и во время сдачи лабораторной работы могут быть заданы и другие, смежные, вопросы.

Список литературы

1. Meta Platforms, Inc. *Meta XR Core SDK Documentation*. <https://developers.meta.com/horizon/documentation/unity/>
2. Fette I., Melnikov A. *The WebSocket Protocol*. RFC 6455, 2011. <https://datatracker.ietf.org/doc/html/rfc6455>
3. PaperMC Team. *Paper Plugin API Reference*. <https://docs.papermc.io/>
4. Mur-Artal R., Tardos J. D. *ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras*. IEEE Transactions on Robotics, 2017.